

Programming Assignment: Unit 7

Hset Paing Htoo

Introduction to Computer Science Program, University of the People

CS 1102-01 Programming 1 AY2026-T5

Instructor : Sirajo Musa (Instructor)

Aug 5, 2026

1. Introduction

For this assignment I built a Student Management System with a graphical user interface using Java Swing. The application lets an administrator add and update student records, enroll students in courses, and assign grades, with the interface updating immediately after every change. I developed and tested the program in Visual Studio Code rather than Eclipse, since Swing is part of the standard Java SDK and has no dependency on any particular IDE -- only a JDK (I used JDK 21) is required to compile and run it.

2. GUI Design

The application opens as a single JFrame titled "Student Management System", with a JMenuBar containing a File menu (with an Exit item) and a JTabbedPane that splits the functionality into three tabs: Students, Enrollment, and Grades. Splitting the interface this way keeps each area of functionality on its own screen instead of overloading a single window with every control at once.

Every tab follows the same layout convention so the interface stays predictable to navigate: a form area (a JPanel using FlowLayout) for input controls, a data display area in the center (a JTable or JList wrapped in a JScrollPane so it scrolls once the data grows), and, on the Grades tab, a second form area at the bottom for assigning a grade. Reusing the same BorderLayout structure (NORTH / CENTER / SOUTH) across all three panels means a user who learns one tab already understands the other two.

3. Implementation Using Java's GUI Framework (Swing)

The application is built entirely with javax.swing and java.awt, and is organized into three layers so that GUI code and data never mix directly:

- Model classes (Student, Course) are plain data holders with no GUI code.
- Manager classes (StudentManager, CourseManager) hold every operation on that data -- adding, updating, looking up, enrolling, grading -- and are the single source of truth the GUI always reads from.
- Panel classes (StudentPanel, EnrollmentPanel, GradePanel) contain only Swing components and event handlers. They never hold a List<Student> or similar collection themselves -- they always ask the relevant manager for the current data.

This separation is what makes the "dynamic interface updates" requirement straightforward: because a panel never keeps its own private copy of the data, it can always rebuild its display by simply asking the manager for the current state again.

MainFrame.java is where everything is assembled -- it creates the two manager instances, builds the tabbed pane, and wires the three panels together:

```
package StudentManagementSystem.src;

import javax.swing.*;
import java.awt.Component;

public class MainFrame extends JFrame {
    public MainFrame() {
        super("Student Management System");

        StudentManager studentManager = new StudentManager();
        CourseManager courseManager = new CourseManager();

        courseManager.addCourse(new Course("CS101", "Intro to Programming"));
        courseManager.addCourse(new Course("CS102", "Data Structures"));

        JTabbedPane tabs = new JTabbedPane();
        tabs.addTab("Students", new StudentPanel(studentManager));

        EnrollmentPanel enrollmentPanel = new EnrollmentPanel(studentManager, courseManager);
```

```

        tabs.addTab("Enrollment", enrollmentPanel);

        GradePanel gradePanel = new GradePanel(studentManager, courseManager);
        tabs.addTab("Grades", gradePanel);

        // Refresh the active tab's data every time the user switches into
        // it, so changes made on other tabs are always reflected.
        tabs.addChangeListener(e -> {
            Component selectedTab = tabs.getSelectedComponent();
            if (selectedTab == gradePanel) {
                gradePanel.refreshStudentList();
            } else if (selectedTab == enrollmentPanel) {
                enrollmentPanel.refreshEligibleList();
            }
        });

        setJMenuBar(buildMenuBar());
        setContentPane(tabs);
        setSize(750, 480);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }

    private JMenuBar buildMenuBar() {
        JMenuBar menuBar = new JMenuBar();
        JMenu fileMenu = new JMenu("File");
        JMenuItem exitItem = new JMenuItem("Exit");
        exitItem.addActionListener(e -> System.exit(0));
        fileMenu.add(exitItem);
        menuBar.add(fileMenu);
        return menuBar;
    }
}

```

MainFrame.java -- top-level window, menu bar, and tab assembly

4. Student Management Functionality

The Students tab provides a form with ID, Name, and Email text fields, alongside "Add Student" and "Update Student" buttons, with a JTable below listing every student currently in the system.

Add Student validates that no field is left empty and that the entered ID is not already in use, showing an error dialog in either case. On success it calls `StudentManager.addStudent()` and immediately rebuilds the table. Update Student looks up the student by the entered ID; if no such student exists, it shows an error dialog rather than silently doing nothing, otherwise it updates that student's name and email and refreshes the table.

Student.java -- the model class for a single student record

```
package StudentManagementSystem.src;

/**
 * Represents a single student record.
 * Kept as a plain data holder (a "model") -- no GUI code belongs in here.
 */
public class Student {
    private String id;
    private String name;
    private String email;

    public Student(String id, String name, String email) {
        this.id = id;
        this.name = name;
        this.email = email;
    }

    public String getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public String getEmail() {
        return email;
    }

    public void setName(String name) {
        this.name = name;
    }

    public void setEmail(String email) {
        this.email = email;
    }
}
```

```

@Override
public String toString() {
    return "(" + id + ") " + name;
}
}

```

Relevant methods from StudentManager.java

```

public class StudentManager {
    private List<Student> students = new ArrayList<>();

    public void addStudent(Student s) {
        students.add(s);
    }

    public Student findById(String id) {
        for (Student student : students) {
            if (student.getId().equals(id))
                return student;
        }
        return null;
    }

    public boolean updateStudent(String id, String newName, String newEmail) {
        Student student = findById(id);
        if (student == null)
            return false;
        student.setName(newName);
        student.setEmail(newEmail);
        return true;
    }

    public List<Student> getAllStudents() {
        return students;
    }

    // ... enrollment and grade methods shown in their own sections below
}

```

StudentPanel.java -- the full Students tab

```

package StudentManagementSystem.src;

import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;

public class StudentPanel extends JPanel {
    private StudentManager manager;
    private DefaultTableModel tableModel;
    private JTable table;

    private JTextField idField = new JTextField(8);
    private JTextField nameField = new JTextField(12);
    private JTextField emailField = new JTextField(14);

    public StudentPanel(StudentManager manager) {
        this.manager = manager;
        setLayout(new BorderLayout(10, 10));
        setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));

        // ---- Form (top) ----
        JPanel formPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
        formPanel.add(new JLabel("ID:"));
        formPanel.add(idField);
        formPanel.add(new JLabel("Name:"));
        formPanel.add(nameField);
        formPanel.add(new JLabel("Email:"));
        formPanel.add(emailField);
    }
}

```

```

JButton addButton = new JButton("Add Student");
addButton.addActionListener(e -> addStudent());
formPanel.add(addButton);

JButton updateButton = new JButton("Update Student");
updateButton.addActionListener(e -> {
    String id = idField.getText().trim();
    String name = nameField.getText().trim();
    String email = emailField.getText().trim();

    if (id.isEmpty() || name.isEmpty() || email.isEmpty()) {
        JOptionPane.showMessageDialog(this,
            "All fields are required.", "Invalid Input",
            JOptionPane.ERROR_MESSAGE);
        return;
    }
    if (manager.findById(id) == null) {
        JOptionPane.showMessageDialog(this,
            "No student with that ID exists.", "Student Not Found",
            JOptionPane.ERROR_MESSAGE);
        return;
    }
    manager.updateStudent(id, name, email);
    refreshTable();
    idField.setText("");
    nameField.setText("");
    emailField.setText("");
});
formPanel.add(updateButton);

add(formPanel, BorderLayout.NORTH);

// ---- Table (center) ----
tableModel = new DefaultTableModel(new Object[] { "ID", "Name", "Email" }, 0) {
    @Override
    public boolean isCellEditable(int row, int col) {
        return false;
    }
};
table = new JTable(tableModel);
add(new JScrollPane(table), BorderLayout.CENTER);
}

private void addStudent() {
    String id = idField.getText().trim();
    String name = nameField.getText().trim();
    String email = emailField.getText().trim();

    if (id.isEmpty() || name.isEmpty() || email.isEmpty()) {
        JOptionPane.showMessageDialog(this,
            "All fields are required.", "Invalid Input",
            JOptionPane.ERROR_MESSAGE);
        return;
    }
    if (manager.findById(id) != null) {
        JOptionPane.showMessageDialog(this,
            "A student with that ID already exists.", "Duplicate ID",
            JOptionPane.ERROR_MESSAGE);
        return;
    }

    manager.addStudent(new Student(id, name, email));
    refreshTable();
    idField.setText("");
    nameField.setText("");
    emailField.setText("");
}

```

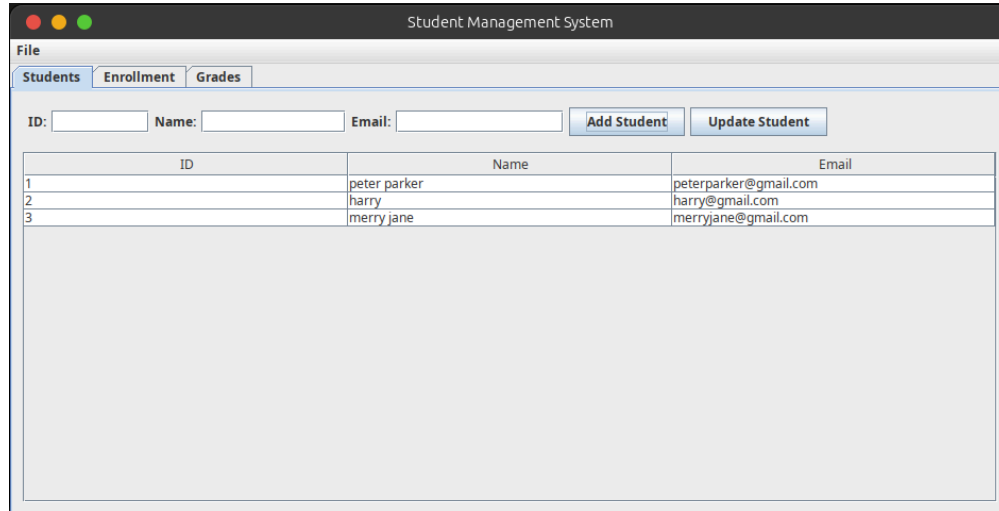
```

    }

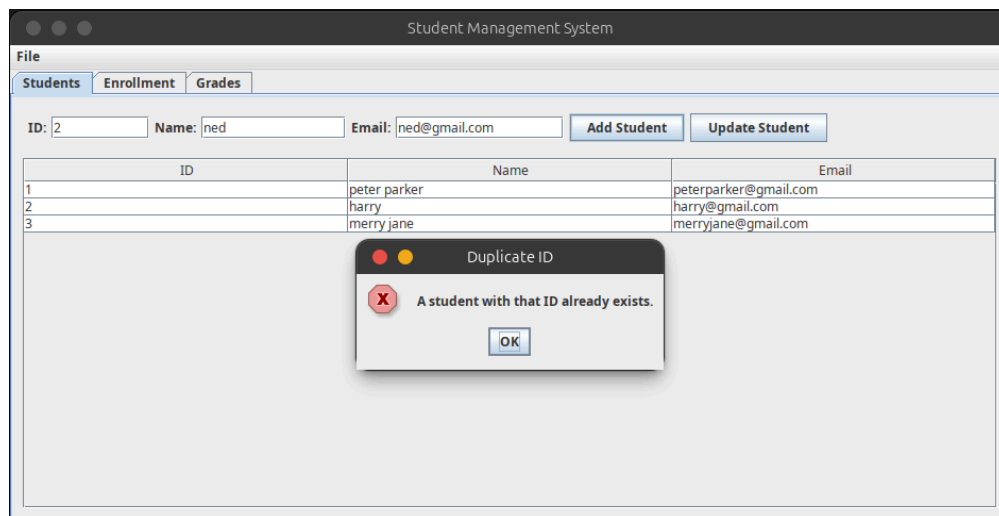
    private void refreshTable() {
        tableModel.setRowCount(0);
        for (Student s : manager.getAllStudents()) {
            tableModel.addRow(new Object[] { s.getId(), s.getName(), s.getEmail() });
        }
    }
}

```

Output



Students tab after adding three students.



Error dialog shown when attempting to add a student with a duplicate ID.

Student Management System

File

Students Enrollment Grades

ID: Name: Email:

ID	Name	Email
1	peter parker	peterparker@gmail.com
2	ned	ned@gmail.com
3	merry jane	merryjane@gmail.com

Table updates immediately after renaming student #2 to "ned" via Update Student.

5. Course Enrollment Functionality

The Enrollment tab has a course dropdown (a `JComboBox<Course>`) at the top and a list of eligible students (a `JList<Student>`) in the center, with an Enroll button underneath. Selecting a course rebuilds the eligible list to show only students who are not already enrolled in it, using `StudentManager.getEligibleStudents()`, which filters out anyone already enrolled. Clicking Enroll after selecting both a course and a student calls `StudentManager.enroll()` and immediately removes that student from the visible list, since they are no longer eligible for that course.

One issue I ran into while building this: the eligible list only rebuilt automatically when the combo box selection itself changed, so a student added on the Students tab would not appear if the user switched to the Enrollment tab afterward without reselecting a course. I fixed this by adding a `refreshEligibleList()` method that `MainFrame` calls whenever the Enrollment tab becomes the active tab (via a `ChangeListener` on the `JTabbedPane`) -- the same "re-read the current state" idea used everywhere else, just triggered by tab navigation instead of a button click.

Course.java -- the model class for a course

```
package StudentManagementSystem.src;

public class Course {
    private String id;
    private String name;

    public Course(String id, String name) {
        this.id = id;
        this.name = name;
    }

    public String getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    @Override
    public String toString() {
        return id + " - " + name;
    }
}
```

CourseManager.java

```
package StudentManagementSystem.src;
```

```

import java.util.ArrayList;
import java.util.List;

public class CourseManager {
    private List<Course> courses = new ArrayList<>();

    public void addCourse(Course c) {
        courses.add(c);
    }

    public Course findById(String id) {
        for (Course c : courses) {
            if (c.getId().equals(id))
                return c;
        }
        return null;
    }

    public Course findByName(String name) {
        for (Course c : courses) {
            if (c.getName().equals(name))
                return c;
        }
        return null;
    }

    public List<Course> getAllCourses() {
        return courses;
    }
}

```

Enrollment-related methods from StudentManager.java

```

// "Who's enrolled where?" tracker, inside StudentManager
private Map<String, List<String>> enrollments = new HashMap<>(); // studentId -> courseIds

public void enroll(String studentId, String courseId) {
    enrollments.computeIfAbsent(studentId, k -> new ArrayList<>()).add(courseId);
}

public boolean isEnrolled(String studentId, String courseId) {
    return enrollments.containsKey(studentId) &&
        enrollments.get(studentId).contains(courseId);
}

public List<Student> getEligibleStudents(String courseId) {
    List<Student> eligible = new ArrayList<>();
    for (Student student : students) {
        if (!isEnrolled(student.getId(), courseId)) {
            eligible.add(student);
        }
    }
    return eligible;
}

```

EnrollmentPanel.java -- the full Enrollment tab

```

package StudentManagementSystem.src;

import javax.swing.*;
import java.awt.*;

public class EnrollmentPanel extends JPanel {
    private StudentManager studentManager;
    private CourseManager courseManager;
    private JComboBox<Course> courseBox;
    private DefaultListModel<Student> listModel;
}

```

```

public EnrollmentPanel(StudentManager studentManager, CourseManager courseManager) {
    this.studentManager = studentManager;
    this.courseManager = courseManager;
    setLayout(new BorderLayout(10, 10));
    setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));

    JPanel formPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
    formPanel.add(new JLabel("Courses: "));

    // ---- Course ComboBox (top) ----
    courseBox = new JComboBox<>();
    for (Course course : courseManager.getAllCourses()) {
        courseBox.addItem(course);
    }
    formPanel.add(courseBox);

    // ---- Eligible Students List (center) ----
    listModel = new DefaultListModel<>();
    JList<Student> eligibleList = new JList<>(listModel);

    courseBox.addActionListener(e -> {
        Course selected = (Course) courseBox.getSelectedItem();
        listModel.clear();
        if (selected != null) {
            for (Student student : studentManager.getEligibleStudents(selected.getId())) {
                listModel.addElement(student);
            }
        }
    });

    add(formPanel, BorderLayout.NORTH);
    add(new JScrollPane(eligibleList), BorderLayout.CENTER);

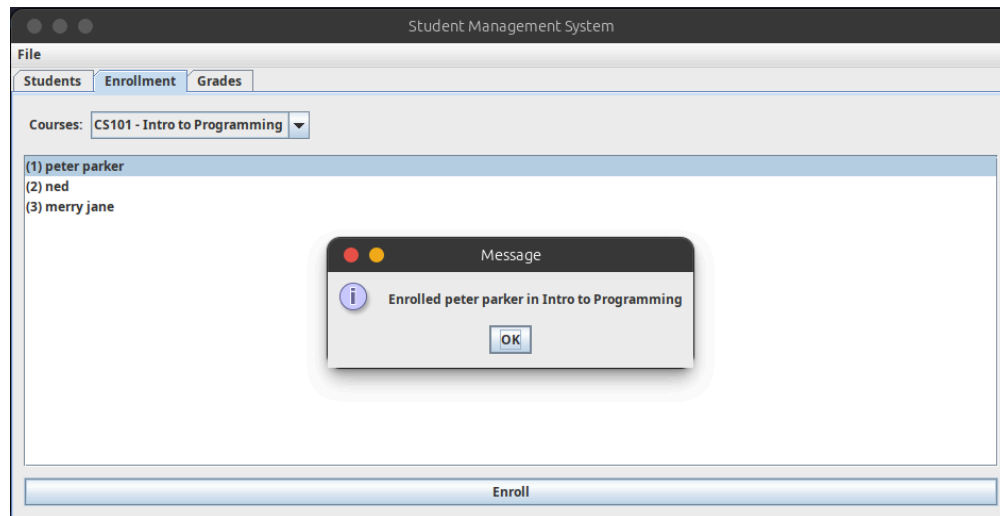
    // ---- Enroll Button (bottom) ----
    JButton enrollButton = new JButton("Enroll");
    enrollButton.addActionListener(e -> {
        Course selectedCourse = (Course) courseBox.getSelectedItem();
        Student selectedStudent = eligibleList.getSelectedValue();
        if (selectedCourse != null && selectedStudent != null) {
            boolean isEnrolled = studentManager.isEnrolled(selectedStudent.getId(),
selectedCourse.getId());
            if (!isEnrolled) {
                studentManager.enroll(selectedStudent.getId(), selectedCourse.getId());
                JOptionPane.showMessageDialog(this,
                    "Enrolled " + selectedStudent.getName() + " in " +
selectedCourse.getName());
                listModel.removeElement(selectedStudent);
            } else {
                JOptionPane.showMessageDialog(this, "Failed to enroll student.", "Error",
                    JOptionPane.ERROR_MESSAGE);
            }
        }
    });
    add(enrollButton, BorderLayout.SOUTH);
}

public void refreshEligibleList() {
    Course selectedCourse = (Course) courseBox.getSelectedItem();
    listModel.clear();
    if (selectedCourse != null) {
        for (Student student : studentManager.getEligibleStudents(selectedCourse.getId()))
        {
            listModel.addElement(student);
        }
    }
}
}

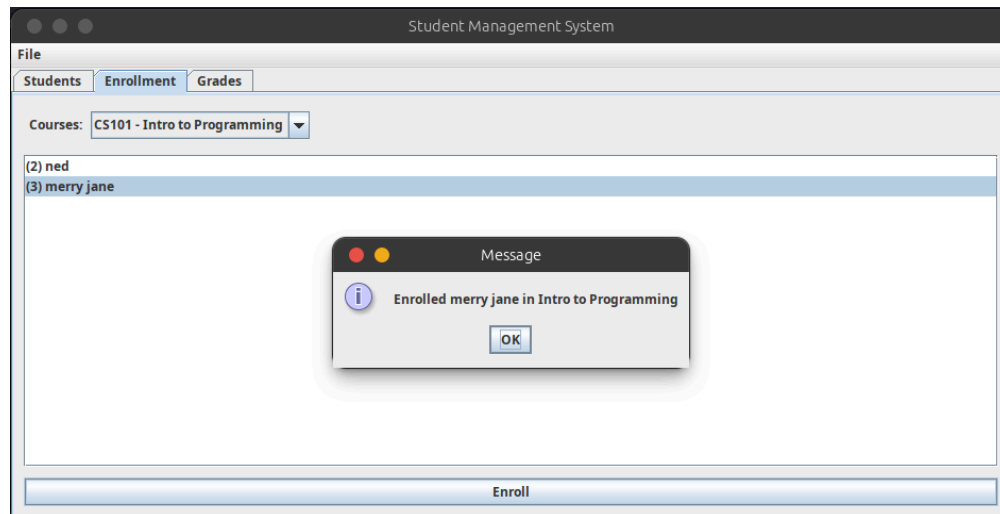
```

}

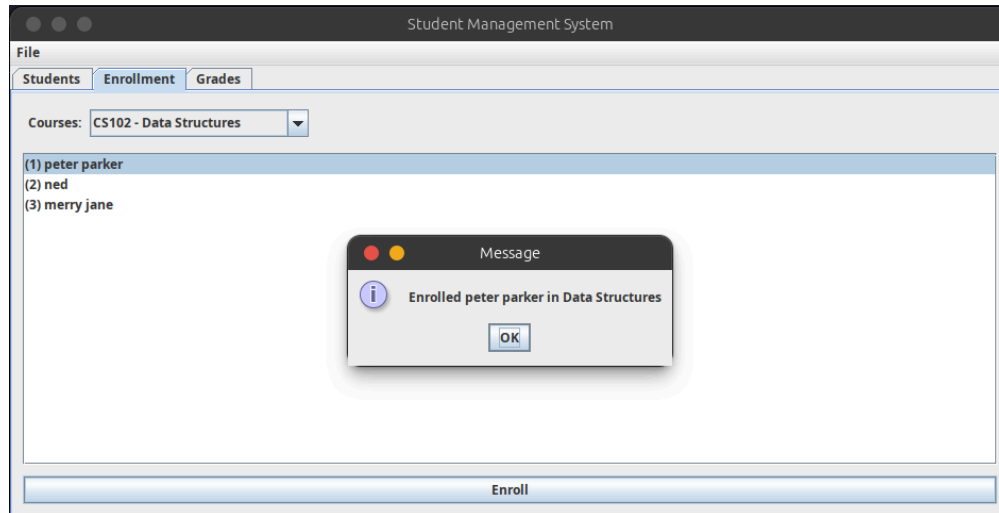
Output



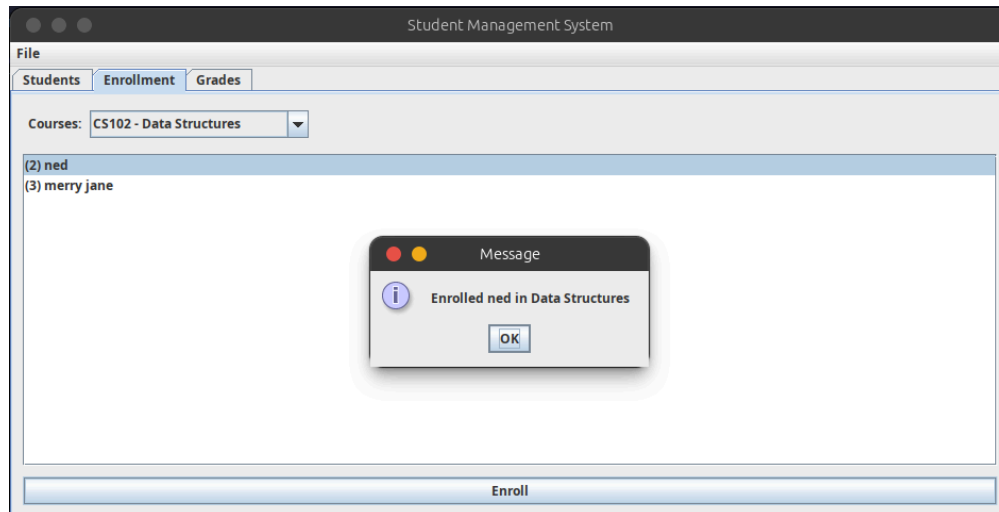
Enrolling "peter parker" into CS101 - Intro to Programming.



After enrolling peter parker, the eligible list for CS101 updates immediately; "merry jane" is enrolled next.



Switching to CS102 - Data Structures shows all three students still eligible, since enrollment is tracked per course.



Enrolling "ned" into CS102 - Data Structures.

6. Grade Management Functionality

The Grades tab has a student dropdown at the top, a table in the center listing that student's enrolled courses alongside their current grade (shown as "-" if ungraded), and a grade-assignment control at the bottom -- a JComboBox of letter grades (A through F) plus an Assign Grade button. Selecting a student rebuilds the table from `StudentManager.getEnrolledCourses()` and `getGrade()`. Assigning a grade requires both a student to be selected in the dropdown and a row of the table to be selected (identifying which course); if either is missing, an error dialog explains what still needs to be selected. On success, `StudentManager.assignGrade()` is called and the table refreshes to show the new grade.

Two bugs came up while building this tab that were worth understanding rather than just patching:

- The student dropdown was only populated once, inside the constructor, so a student added afterward on the Students tab never appeared in it. This needed the same tab-change-triggered refresh used for the Enrollment tab -- `MainFrame` calls `refreshStudentList()` every time the Grades tab becomes active.
- I initially had the student dropdown's own selection listener call that same refresh method. Repopulating a JComboBox can itself fire another selection-changed event, so the listener re-entered itself in the middle of populating and ended up adding one student twice. The fix was to keep "populate this dropdown" and "react to a selection in this dropdown" as two separate methods that are never called from within each other.

Grade-related methods from StudentManager.java

```
// Grades tracker, inside StudentManager
private Map<String, Map<String, String>> grades = new HashMap<>(); // studentId -> (courseId
-> grade)

public void assignGrade(String studentId, String courseId, String grade) {
    grades.computeIfAbsent(studentId, k -> new HashMap<>()).put(courseId, grade);
}

public String getGrade(String studentId, String courseId) {
    if (grades.containsKey(studentId) && grades.get(studentId).containsKey(courseId)) {
        return grades.get(studentId).get(courseId);
    }
    return null;
}
```

```

public List<String> getEnrolledCourses(String studentId) {
    return enrollments.getOrDefault(studentId, new ArrayList<>());
}

```

GradePanel.java -- the full Grades tab

```

package StudentManagementSystem.src;

import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;

public class GradePanel extends JPanel {
    private StudentManager studentManager;
    private CourseManager courseManager;
    private DefaultTableModel tableModel;
    private JTable table;
    private JComboBox<Student> studentBox;

    public GradePanel(StudentManager studentManager, CourseManager courseManager) {
        this.studentManager = studentManager;
        this.courseManager = courseManager;
        setLayout(new BorderLayout(10, 10));
        setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));

        JPanel formPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
        formPanel.add(new JLabel("Students: "));

        // ---- Student ComboBox (top) ----
        studentBox = new JComboBox<>();
        for (Student student : studentManager.getAllStudents()) {
            studentBox.addItem(student);
        }
        formPanel.add(studentBox);
        add(formPanel, BorderLayout.NORTH);

        studentBox.addActionListener(e -> {
            Student selectedStudent = (Student) studentBox.getSelectedItem();
            refreshTable(selectedStudent);
        });

        // ---- Courses & Grades Table (center) ----
        tableModel = new DefaultTableModel(new Object[] { "Course Name", "Grade" }, 0) {
            @Override
            public boolean isCellEditable(int row, int col) {
                return false;
            }
        };
        table = new JTable(tableModel);
        add(new JScrollPane(table), BorderLayout.CENTER);

        // ---- Grade Assignment (bottom) ----
        JComboBox<String> gradeBox = new JComboBox<>(new String[] { "A", "B", "C", "D", "F"
    });

    JButton assignButton = new JButton("Assign Grade");

    JPanel assignPanel = new JPanel(new FlowLayout(FlowLayout.CENTER));
    assignPanel.add(new JLabel("Grade:"));
    assignPanel.add(gradeBox);
    assignPanel.add(assignButton);

    assignButton.addActionListener(e -> {
        Student selectedStudent = (Student) studentBox.getSelectedItem();
        int selectedRow = table.getSelectedRow();
        if (selectedStudent != null && selectedRow != -1) {
            String courseName = (String) tableModel.getValueAt(selectedRow, 0);

```



```

        Course course = courseManager.findByName(courseName);
        if (course != null) {
            String selectedGrade = (String) gradeBox.getSelectedItemAt(
selectedGrade);
            studentManager.assignGrade(selectedStudent.getId(), course.getId(),
refreshTable(selectedStudent);
        }
    } else {
        JOptionPane.showMessageDialog(this, "Please select a student and a course.",
>Error",
        JOptionPane.ERROR_MESSAGE);
    }
});

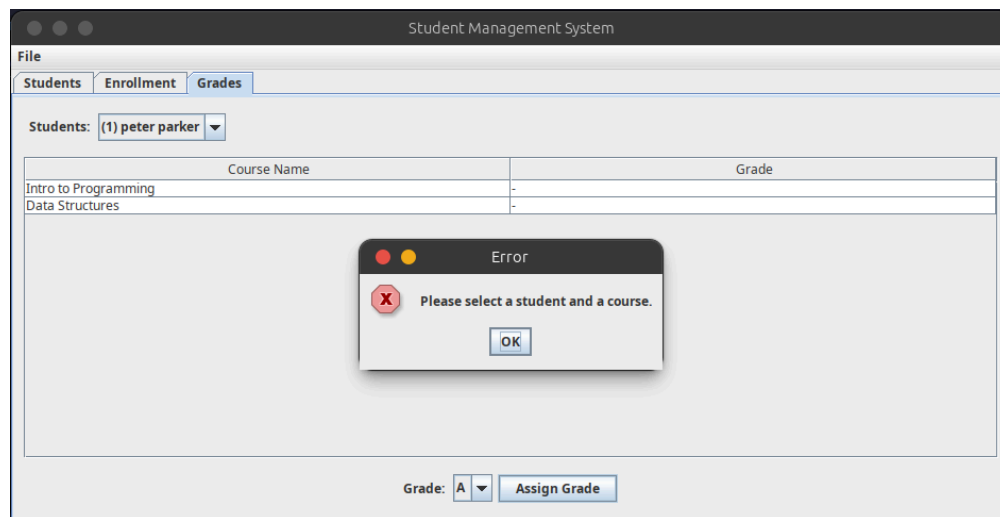
add(assignPanel, BorderLayout.SOUTH);
}

public void refreshStudentList() {
    Student selectedStudent = (Student) studentBox.getSelectedItemAt(
studentBox.removeAllItems();
    for (Student student : studentManager.getAllStudents()) {
        studentBox.addItem(student);
    }
    if (selectedStudent != null) {
        studentBox.setSelectedItem(selectedStudent);
    }
}

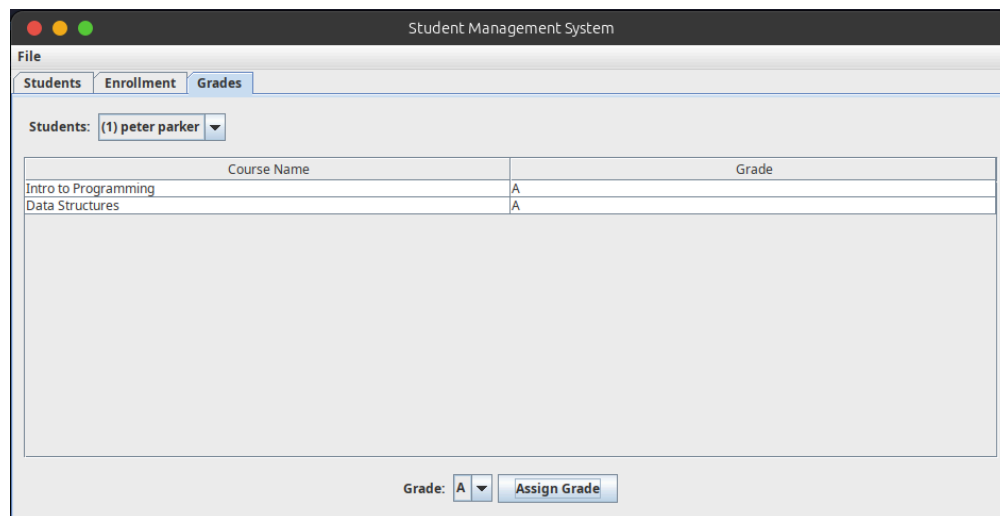
private void refreshTable(Student student) {
    tableModel.setRowCount(0);
    if (student != null) {
        for (String courseId : studentManager.getEnrolledCourses(student.getId())) {
            Course course = courseManager.findById(courseId);
            String grade = studentManager.getGrade(student.getId(), courseId);
            tableModel.addRow(new Object[] { course.getName(), grade != null ? grade : "-"
});
        }
    }
}
}

```

Output



Error dialog shown when trying to assign a grade without selecting both a student and a course row.



Grades of A assigned to peter parker for both enrolled courses.

Student Management System

File

Students Enrollment Grades

Students: (2) ned ▼

Course Name	Grade
Data Structures	A

Grade: A ▼ Assign Grade

Grade for "ned", showing only the one course ned is actually enrolled in.

Student Management System

File

Students Enrollment Grades

Students: (3) merry jane ▼

Course Name	Grade
Intro to Programming	A

Grade: A ▼ Assign Grade

Grade for "merry jane".

7. Dynamic Interface Updates

Every panel follows the same rule: after any change to data (adding or updating a student, enrolling a student, assigning a grade), the panel immediately re-reads the current state from the relevant manager and rebuilds its table or list from scratch, rather than trying to patch the existing display in place. This is what `refreshTable()` does in `StudentPanel` and `GradePanel`, and what `refreshEligibleList()` does in `EnrollmentPanel`.

For the two dropdown-driven tabs, this alone was not enough, since a combo-box-driven refresh only fires when the selection itself changes, not when the underlying data changes elsewhere. I extended the pattern with a `ChangeListener` on the main `JTabbedPane` in `MainFrame`, so that simply switching into the Enrollment or Grades tab also triggers a refresh -- catching cases like a new student added on the Students tab while the Grades tab was not visible:

```
tabs.addChangeListener(e -> {  
    Component selectedTab = tabs.getSelectedComponent();  
    if (selectedTab == gradePanel) {  
        gradePanel.refreshStudentList();  
    } else if (selectedTab == enrollmentPanel) {  
        enrollmentPanel.refreshEligibleList();  
    }  
});
```

Tab-change listener in `MainFrame.java` that keeps every tab's data current

8. Error Handling

Every form-based action validates its inputs before changing any data, and reports problems through `JOptionPane.showMessageDialog` with `JOptionPane.ERROR_MESSAGE`, rather than failing silently or letting an invalid state reach the data layer. The specific cases handled are:

- Empty required fields when adding or updating a student.
- Adding a student with an ID that already exists (Duplicate ID).
- Updating a student ID that does not exist (Student Not Found).

- Attempting to enroll a student already enrolled in the selected course.
- Attempting to assign a grade without both a student and a course row selected.

Screenshots demonstrating two of these cases (Duplicate ID in Section 4, and the missing-selection error in Section 6) are shown above alongside the functionality they belong to.

9. How to Run the Program

- Requires a JDK (developed and tested against JDK 21).
- Open the project folder (containing the src directory) in any Java-capable editor -- developed in VS Code with the "Extension Pack for Java" extension.
- Open Main.java and run it: in VS Code, click the Run link above the main method; from a terminal, run "`javac -d out $(find src -name '*.java')`" followed by "`java -cp out StudentManagementSystem.src.Main`" from the project root.
- The application opens on the Students tab. Use the Students, Enrollment, and Grades tabs to navigate between functionality, and either the File menu's Exit item or the window's close button to quit.

Appendix: Main.java

The application's entry point, which starts the GUI on Swing's event dispatch thread as required for thread safety:

```
package StudentManagementSystem.src;

import javax.swing.SwingUtilities;

public class Main {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new MainFrame().setVisible(true));
    }
}
```